Bachelor of Computer Applications (Hons. Game Development)
Semester- III

Subject: Database Management System

Assignment 4: Relational Database vs. NoSQL Database

By
Name: FIDHA
Reg. No.: 2511021240021

Faculty Name: Renita Joy
Faculty Signature:_____

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru - 562 106
October 2026

1. Introduction

Relational and NoSQL databases solve overlapping but different data-management problems. The best choice depends on data shape, consistency needs, query patterns, operational skills, and growth requirements. The Library Management System in this project is naturally relational because it contains structured entities and clear foreign-key relationships.

2. Comparison

Feature	Relational Database (RDBMS)	NoSQL Database
Scalability	• Vertical scaling (bigger servers) • Can scale horizontally, but is more complex	• Built for horizontal scaling • Easily handles large volumes of data across multiple servers
Consistency	• Strong consistency (ACID properties) • Ensures data accuracy and integrity	• Eventual consistency (in most cases) • May have slight delays in data propagation (depends on the database type)
Performance	• Very good for complex queries • Performance can decrease with very large data volumes or high traffic	• High performance for read/write operations • Optimized for specific use cases (e.g., document, key-value, column, graph)
Data Model	• Structured (tables, rows, columns) • Fixed schema	• Flexible (document, key-value, column, graph) • Schema-less or dynamic schema
Complex Queries	• Excellent support for complex joins, aggregations and transactions	• Limited or no support for joins • Simpler queries, optimized for specific patterns
Best Suited For	• Financial systems, e-commerce, ERP, CRM, transaction-heavy applications	• Big data, real-time apps, social media, content management, IoT, gaming

Real-World Examples (RDBMS)

- **MySQL** (web applications, e-commerce)
- **PostgreSQL** (enterprise systems, analytics)
- **Oracle** (large enterprises, transaction systems)
- **SQL Server** (business applications, Windows ecosystem)

Real-World Examples (NoSQL)

- **MongoDB** (content management, e-commerce)
- **Cassandra** (big data, logging, real-time analytics)
- **Redis** (caching, real-time applications)
- **Neo4j** (graph data, social networks)

3. Real-world examples

Relational examples: MySQL and PostgreSQL are commonly used for business systems with structured records and transactions. A library system can use foreign keys to ensure that a loan references an existing member, staff member, and physical book copy. Financial and order-processing systems also commonly use relational transactions to keep related updates consistent.

NoSQL examples: MongoDB is a document database suited to flexible document-shaped records; Redis is a key-value store commonly used for caching and fast ephemeral lookups; Apache Cassandra is a wide-column database designed for distributed, high-write workloads; Neo4j is a graph database suited to highly connected data such as recommendation or network relationships. These examples describe common fit, not a claim that each organization uses only that technology.

4. Why organizations choose different technologies

Organizations choose based on requirements rather than on a database being universally superior. A relational database is a good default when relationships, constraints, and multi-table transactions are central. A NoSQL database may be appropriate when the workload benefits from flexible records, specific access patterns, large distributed write volume, or graph traversal. Some systems use multiple databases for different workloads, a pattern called polyglot persistence.

5. Recommendation for the proposed Library Management System

Use a relational database such as MySQL for the current project. Members, Books, BookCopies, Loans, Reservations, and Fines have clear relationships and require referential integrity. SQL joins also make the required circulation, reservation, and fine reports straightforward. A NoSQL store is not necessary for the core system based on the scope provided. It could be considered later for a separate feature with different needs, but the current assignment does not require it.

6. Conclusion

Relational databases prioritize structured relationships and transactional integrity; NoSQL databases offer several alternative models that can suit particular scaling and data-shape needs. The database should be selected to fit the application's actual workload, consistency requirements, and team capabilities.